

AI Product Copilot

Turn ambiguous product ideas into structured, reviewed, and human-approved product requirements.

LOCAL-FIRST AI PRODUCT WORKFLOW

AI Product Copilot

Turn ambiguous product ideas into structured, reviewed, and human-approved product requirements. Built to explore how AI can assist product decisions without replacing product judgment.

Qwen3:8B

Ollama

LangChain

Structured Output

Human-in-the-loop

Local-first

01 / DEFINE	02 / ANALYZE	03 / REVIEW	04 / DECIDE
Define	Analyze	Review	Decide
Describe an ambiguous product idea.	Generate structured requirements.	Detect assumptions and quality risks.	Human input shapes the final PRD.

AI Product Copilot is a local-first AI application that helps product managers transform early-stage product ideas into structured product analysis and an MVP Product Requirements Document (PRD).

Instead of treating LLM output as ground truth, the application combines **structured generation, AI quality review, and human product judgment** before producing the final artifact.

Built with **Qwen3:8B, Ollama, LangChain, Pydantic, and Streamlit**.

Demo

AI Product Copilot uses a four-stage workflow:

Define → Analyze → Review → Decide

The system first converts an ambiguous idea into structured product requirements.

01 / DEFINE

What are you trying to build?

Build an AI resume screening feature that helps recruiters identify candidates who match a job description.

Analyze Product Idea →

Local AI
Qwen3:8B runs locally through Ollama. Product analysis is returned as validated structured output rather than free-form chat.

02 / ANALYZE

Structured Product Analysis

Problem

Recruiters spend significant time manually screening resumes to identify candidates matching job requirements, leading to inefficiencies and potential oversight of qualified applicants.

Target Users

- Recruiters
- HR professionals
- Talent acquisition specialists

User Stories

- **As a Recruiter**, I want to screen resumes for job fit, so that save time on manual screening.
- **As a Recruiter**, I want to compare candidate skills to job requirements, so that identify top candidates more efficiently.
- **As a Recruiter**, I want to validate AI recommendations, so that reduce risk of hiring unqualified candidates.

MVP Features

- AI-powered resume parsing and keyword matching
- Job description analysis to identify key requirements
- Candidate-job match scoring system
- Dashboard for reviewing top matches
- Exportable shortlist of candidates

Acceptance Criteria

- System correctly identifies and highlights job description keywords in resumes
- Candidates are scored based on alignment with job requirements
- Recruiters can filter candidates by score thresholds
- System handles common resume formats (PDF, Word, etc.)
- Matches are visually distinct from non-matching candidates

> Risks, assumptions & open questions

A second AI pass critiques the analysis for unsupported metrics, assumptions, scope creep, AI risks, and false certainty.

The product manager then reviews those findings and provides explicit product decisions before the final PRD is generated.

03 / REVIEW

AI Quality Review

5 issues detected

A second model pass critiques the generated analysis. Findings are advisory, not ground truth.

- > 1. Unsupported metrics
- > 2. Unsupported assumptions
- > 3. MVP scope creep
- > 4. AI risks
- > 5. False certainty

04 / DECIDE

Human Product Decision

Review the AI output, then add the product decisions that should shape the final PRD.

Product Manager Instructions
from the MVP:

The AI should explain which job requirement each resume appears to match and cite supporting evidence from the resume.

Recruiters must make the final shortlist decision.

Do not assume ATS integration.

Candidate privacy should be treated as a key product risk.

Generate Human-Reviewed PRD →

Generating human-reviewed PRD...

Why human review?

The quality reviewer is another LLM, not an authority.

Human instructions therefore take priority when the final PRD is generated. This keeps consequential product decisions under human control.

The final PRD incorporates human decisions while preserving identified risks and open questions.

OUTPUT

Final Product Requirements Document

Generated from the original idea, structured analysis, AI critique, and human product decisions.

AI Resume Matching Assistant MVP

Executive Summary

An AI tool that helps recruiters understand how resumes align with job requirements by highlighting specific matches and evidence from resumes. Recruiters retain full control over final shortlist decisions.

Problem Statement

Recruiters spend significant time manually screening resumes to match job descriptions, leading to inefficiency and potential bias in candidate selection.

Target Users

- Recruiters at mid-sized to large organizations
- HR professionals with high-volume hiring needs

Goals

- Enable recruiters to identify resume-job alignment with AI assistance
- Maintain human control over final hiring decisions
- Ensure candidate privacy and data security

User Stories

- As a recruiter, I want the AI to show which parts of the resume match specific job requirements so I can quickly assess relevance.
- As a recruiter, I want the AI to cite specific phrases from the resume that support a match so I can verify alignment.
- As a recruiter, I want to make the final decision about which candidates to advance so I retain control over the hiring process.

MVP Scope

- Job description upload interface
- Resume parsing and keyword extraction
- AI explanation of resume-job matches with resume evidence
- Candidate review interface with manual shortlist creation

Non-Goals

- Automated candidate ranking or scoring
- ATS system integration
- Exportable shortlists
- Predictive hiring analytics

Acceptance Criteria

- System displays job requirements and highlights corresponding resume content
- AI explanations include specific text excerpts from resumes
- Recruiters can manually select candidates for shortlist without AI automation
- System handles resume parsing without requiring ATS integration
- Candidate data is stored securely with access controls

Risks

- Privacy: Handling of sensitive candidate information
- Hallucination: AI may misattribute qualifications not present in resumes
- Bias: Potential for algorithmic bias in matching explanations
- Trust: Recruiters may question AI-generated match explanations

Open Questions

- How to balance AI explanation with recruiter discretion?
- What level of detail is needed for job descriptions to achieve meaningful matches?
- How to handle ambiguous or subjective job requirements?
- What are the long-term maintenance requirements for the AI model?

Download PRD (.md)

The exported document remains a draft. AI-generated requirements should be validated with users, product stakeholders, and engineering before implementation.

AI Product Copilot - Qwen3.8B - Ollama - LangChain - Local-first - Human-in-the-loop

Why I Built This

Large language models can generate convincing product requirements quickly, but convincing output is not necessarily reliable output.

During early testing, the model generated unsupported requirements such as arbitrary accuracy targets, latency targets, and assumptions about user behavior.

For example, acceptance criteria included metrics such as:

- 80% keyword matching
- 5-second processing time

even though those targets were never provided by the user or supported by research.

This led to the core product question behind this project:

How can AI accelerate product requirement development without allowing generated assumptions to silently become product decisions?

AI Product Copilot explores a workflow where AI generates and critiques product requirements, while the product manager retains control over final decisions.

Product Workflow

The application follows four main stages:

1. Define

The product manager enters an ambiguous product idea.

Example:

Build an AI resume screening feature that helps recruiters identify candidates who match a job description.

2. Analyze

Qwen3 generates a structured product analysis including:

- Problem
- Target users
- User stories
- MVP features
- Acceptance criteria
- Risks
- Assumptions
- Open questions

The response is constrained using a **Pydantic schema** instead of relying on free-form text generation.

3. Review

A separate AI review step critiques the generated analysis for potential quality issues such as:

- Unsupported metrics
- Unsupported assumptions
- MVP scope creep
- AI risks

- False certainty
- Missing considerations

The reviewer is intentionally treated as an **advisory system rather than an authority**.

An AI reviewer can also hallucinate or make questionable recommendations.

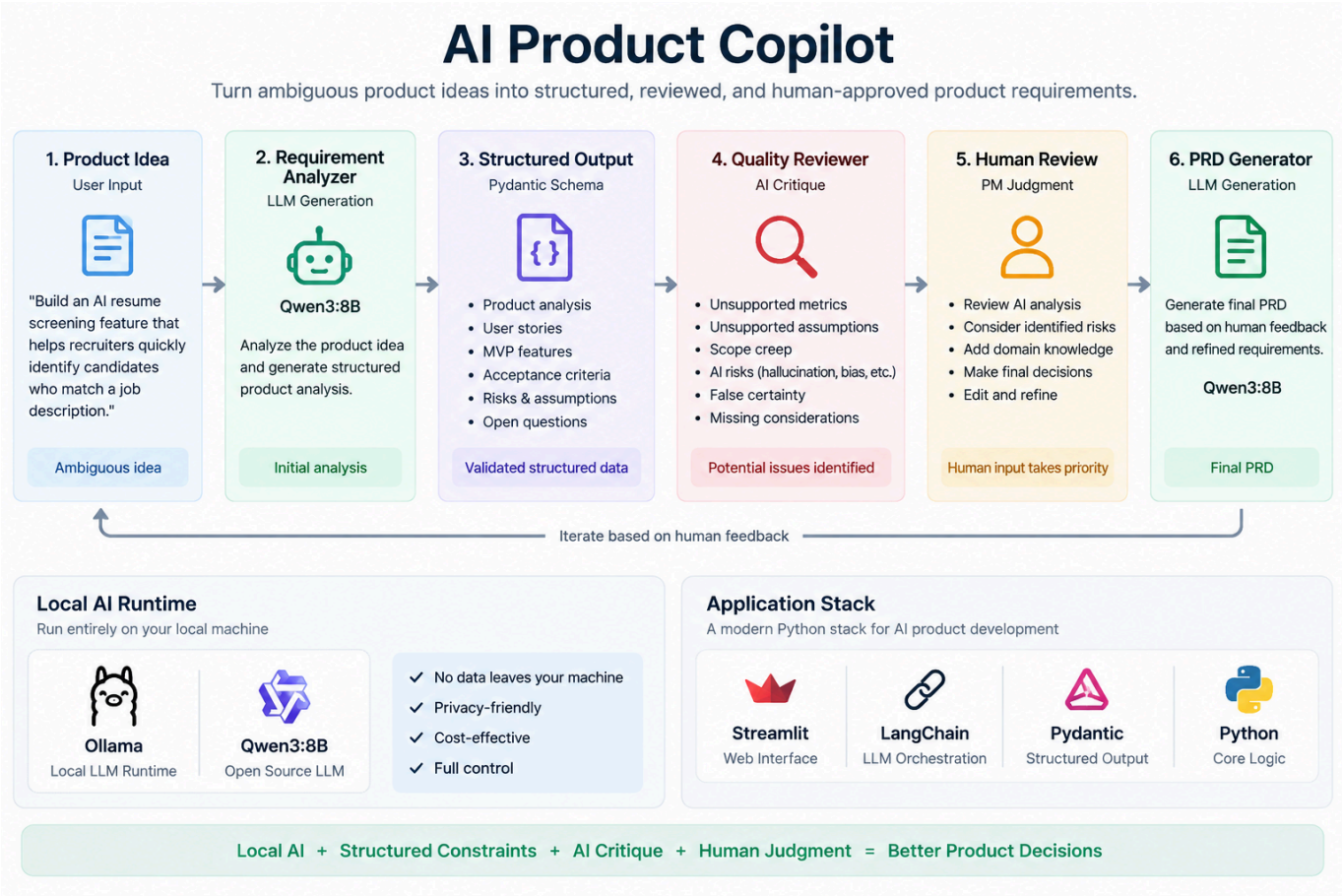
4. Decide

The product manager reviews both the original analysis and the AI critique.

Human instructions can override AI recommendations before the final PRD is generated.

The final document can then be exported as Markdown.

Architecture



```
Human Product Decision
  ↓
PRD Generator
  ↓
Final PRD
```

Local AI Runtime

The application currently runs locally using:

- Ollama
- Qwen3:8B

This makes local experimentation possible without requiring product inputs to be sent to an external hosted LLM API.

Key AI Product Decisions

Structured Output over Free-form Generation

The first prototype returned a Markdown document directly from the LLM.

That worked for a demo, but it made the application dependent on unpredictable model formatting.

The workflow was changed to:

```
LLM
  ↓
Pydantic Schema
  ↓
Validated Application Data
  ↓
UI
```

This separates model generation from presentation and gives the application control over the output structure.

Schema Validation Does Not Guarantee Semantic Quality

Structured output solved formatting reliability, but testing revealed that the model could still produce unsupported content inside a perfectly valid schema.

For example, the model introduced numerical acceptance criteria that were never supplied by the user.

This demonstrated an important distinction:

Structural validity is not the same as factual or product validity.

The Quality Reviewer was added to identify these semantic issues.

AI Reviewer as Advisor, Not Judge

A second LLM pass reviews generated requirements.

However, the reviewer is not assumed to be correct.

During development, the reviewer correctly criticized an unsupported numerical accuracy target, but then suggested another unsupported numerical target in its own recommendation.

This reinforced the decision not to automatically apply reviewer recommendations.

Instead:

```
AI generates
  ↓
AI critiques
  ↓
Human decides
```

Human-in-the-loop

Human product instructions have higher priority than AI reviewer recommendations when generating the final PRD.

For example, a product manager can explicitly specify:

```
Remove automated candidate ranking from the MVP.

Recruiters must make the final shortlist decision.

Show supporting resume evidence for AI-generated matches.

Do not assume ATS integration.

Treat candidate privacy as a key product risk.
```

The PRD generator then incorporates those decisions into the final document.

This keeps consequential product decisions under human control.

Local-first Development

The current implementation uses Qwen3:8B through Ollama.

Reasons for choosing a local model for this prototype include:

- Fast experimentation without API integration
- Local handling of prototype inputs
- No per-request inference API cost
- Ability to explore model behavior directly
- Simple reproducible development environment

The model layer is separated from the product workflow so another LLM provider could be introduced later.

Example: Human Review Changes the PRD

One test case started with an AI-generated resume-screening product concept.

The AI initially proposed:

- Candidate ranking
- Relevance scoring
- Automated filtering

During human review, the product manager instructed the system to:

- Remove automated candidate ranking
- Remove relevance scores
- Require evidence for AI-generated matches
- Keep final shortlist decisions with recruiters
- Treat candidate privacy as a key risk

The final PRD reflected these decisions by moving automated ranking into **Non-Goals**, introducing evidence-based matching, preserving human decision-making, and explicitly identifying privacy risk.

This demonstrates the intended behavior of the system:

AI proposes. AI critiques. Humans decide.

Tech Stack

Layer	Technology
LLM	Qwen3:8B
Local inference	Ollama
LLM orchestration	LangChain
Structured output	Pydantic
Application UI	Streamlit
Language	Python

Project Structure

```
ai-product-copilot/
├── app.py
├── README.md
├── requirements.txt
├── src/
│   ├── __init__.py
│   ├── llm.py
│   ├── prompts.py
│   ├── schemas.py
│   └── workflow.py
├── docs/
│   └── architecture.png
├── screenshots/
└── sample_data/
```

Run Locally

Prerequisites

- Python
- Ollama
- Qwen3:8B

Pull the model if necessary:

```
ollama pull qwen3:8b
```

Clone the repository:

```
git clone <YOUR_REPOSITORY_URL>
cd ai-product-copilot
```

Create a virtual environment:

```
python -m venv .venv
```

Activate it on Windows PowerShell:

```
.\.venv\Scripts\Activate.ps1
```

Install dependencies:

```
pip install -r requirements.txt
```

Start the application:

```
streamlit run app.py
```

Open the local Streamlit URL shown in the terminal.

Current Limitations

This project is an MVP designed to explore an AI-assisted product workflow rather than a production PRD system.

Current limitations include:

- AI-generated analysis can still contain unsupported assumptions.
- The Quality Reviewer is itself an LLM and can produce incorrect recommendations.
- Structured output improves format reliability but does not guarantee factual correctness.
- Acceptance criteria may still contain language that requires human refinement.
- Product recommendations are not grounded in actual user research unless that evidence is provided.
- The current workflow does not yet retrieve supporting evidence from external research or internal knowledge bases.
- Local model performance depends on available hardware.

These limitations are why human review remains part of the workflow.

Potential Next Steps

Future iterations could explore:

- Evidence-grounded product analysis using RAG
- Uploading user interviews and research documents
- Source citations for generated requirements
- Editable structured requirements before PRD generation
- Multiple LLM provider support
- Evaluation datasets for requirement quality
- Prompt and model comparison
- PRD version history
- Collaborative review workflows

Design Principle

AI should accelerate product thinking, not silently replace product judgment.

The goal of AI Product Copilot is therefore not to automatically write the “correct” PRD.

It is to help product managers move faster while making assumptions, risks, AI limitations, and human decisions more visible.